10万+Skill 背后：腾讯SkillHub如何帮用户找到真正好用的那20%

原创 腾讯程序员 腾讯技术工程 Aug 10, 2026, 2:36 AM 广东 985 listened



作者：赵秀雯

导语：AI 时代下，平台如何做内容治理与分发？SkillHub 作为国内最大的 Skill 社区之一，与小红书、知乎、App Store 这些传统内容社区做推广搜有什么不同？SkillHub 是如何帮助用户快速找到那 20% 值得被看见的 Skill 的？本文将通过深度解读 SkillHub 针对这些命题的思考与实践，来逐一为大家解答。



SkillHub 是专为中国用户优化的 AI Skills 社区，自今年 3 月上线以来，跟随 OpenClaw 生态的快速发展获得了广泛关注。作为国内最大的 Skill 社区之一，我们在社群中最常被问到的一个问题是：

平台上那么多 Skill，我怎么知道哪一个真的好用，有推荐的 Skill 吗？

早在3个月前，当时 SkillHub 的数量大概是 5万左右，用户当时已经是挑花眼，不知道哪个 Skill 值得安装。对于一个拥有庞大资产内容的平台来说，供给的数量与供给创造的价值，往往并不均匀。AI 又让这个问题变得更突出。过去，生产本身就有门槛，但在今天，AI 大大降低了用户写作、编码、创作的成本。创作者借助 AI 一句话生成一个 Skill，立马发布到 SkillHub 上，先把 Skill 的唯一标识码抢注了，就跟五百年前抢注域名热潮一样。

就正如咱们常说的二八法则，真正有价值的内容往往只有那 20%。平台快速增长的同时，也迫使我们从早期偏粗放的供给扩充，转向更系统的推荐、分发与治理。**在这个过程中，如何处理好消费者、创作者和平台之间的关系，成为 SkillHub 很关键的问题。**消费者希望更快找

到可信、适合自己的 Skill；创作者希望真正优质的作品能够被公平看见，并获得有效反馈；平台则需要在供给增长、分发效率和社区信任之间保持平衡。

到目前为止，平台已经收录 10 万+个 AI Skills，月下载量超过 1000 万次，数据仍在持续上涨。因此，我想借这篇文章复盘 SkillHub 的经验：在内容快速增长的情况下，作为一个诞生于 AI 时代的 AI Native 社区，SkillHub 如何寻找、验证并放大这部分真正值得被看见的 Skill。

在这个命题上，我们核心做了三件事：

- 建立一套可信的质量坐标；
- 把有限的曝光分给更值得被看见的供给；
- 让这套发现能力不仅服务人，也能被 AI 直接使用，这也是在 AI 时代下，跟传统社区最大的区别。

一、社区的老方法依然有效，SkillHub 还要多回答一道题

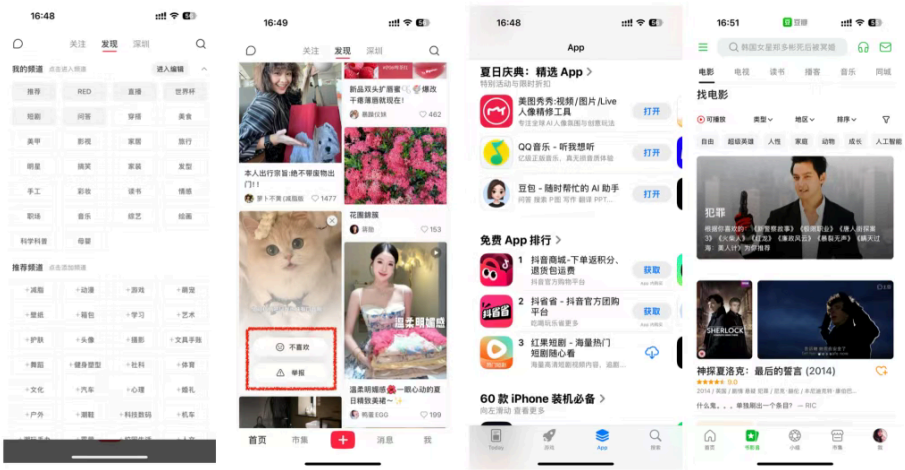
在开始分享 SkillHub 的实践前，可以先回到最朴素的问题：**对于一般的社区来说，是怎么解决内容治理和分发问题的？**那些成熟的社区，在处理海量供给时，通常有以下几种成熟的办法：

第一种，组织供给。通过频道、分类、标签和搜索，把原本平铺的内容放进用户可以理解的结构里。例如豆瓣、知乎。

第二种，分配流量。通过热门榜、新品榜、个性化推荐、编辑精选，把有限的展示位置分配给更可能被消费的内容，回答“先看什么”。例如App Store。

第三种：建立治理门槛。用发布审核、信用认证、举报、降权和下架机制，过滤明显低质或有风险的供给，回答“什么不应该被看见”。例如小红书。

第四种：形成反馈闭环。点击、停留、购买、收藏、评分等行为不断回流，帮助平台判断内容是否真正满足需求，也让创作者知道下一步该优化什么。例如淘宝。



对于 SkillHub 来说，过去这些社区产品方法论，其实依然适用。就像是道路上的交通工具从自行车、燃油车换成了电动车，车变了，但红绿灯、路标、斑马线这些规则依然有效。只是，当我在规划把这套方法搬到 SkillHub 的时候，很快又想到了一个问题：一篇文章被读完、一个商品被购买，平台就能获得相对清晰的消费信号。但 Skill 的真正使用，是用户从 SkillHub 下载安装到 Agent 后才发生的，换句话说，Skill 的反馈其实是有滞后性的。一个 Skill 被点击或下载，不代表它就能成功完成任务。

所以对于 SkillHub 来说，分类、榜单和行为数据仍然重要，但还不够，在讨论“如何推荐”之前，我们必须先补上一层普通社区很少需要深入处理的能力：**这个 Skill 到底好不好，依据是什么。**

二、先建立质量坐标：TRACE 回答“什么是好的 Skill”

如果把 Skill 类比成代码，那么审核 Skill 就很像是在做 Code Review。

最开始面临怎么判断上架后的 Skill 是否值得被推荐这个问题时，我们想到了两个问题，一是如果没有统一的标准，判断容易因人而异，二是平台里越来越多内容供给，如何支撑数万量级的内容持续评估也是值得被考虑的因素。

TRACE 评测体系的起点，就是来自这里：把“这个 Skill 好不好，是否值得信赖”从一种模糊感受，变成可确认可解释的质量坐标。设计这套体系时，一方面我们参考了业内例如 Anthropic 发布的 Skill 规范标准，另一方面，更多是我们对用户痛点的洞察以及对 SkillHub 的定位与思考。

在一开始 SkillHub 出生的时候，当时国内还没有比较成规模的 Skill 社区。而我所在的团队 —— 腾讯轻量云，在今年1月的时候就适配了 OpenClaw 龙虾的云端部署并推出市场，而到3月我们在深圳腾大楼下举办一场公益装机活动，又让龙虾彻底火到了老奶奶人群。当时整个 Agent 和 Skill 生态，还是以国外为主，大家首选还是 ClawHub。但这里就有出现的非常明显的问题：海外的 Skill，未必在国内适用，它可能会存在内容合规性、安全风险以及海外产品/API 在国内无法调用等问题。

因此，首要需要先检测这个 Skill，是否安全、是否适配国内环境，我们也把 Trust 放在了整个测评体系的首要位置。**这也是体现了 SkillHub 的核心定位 —— 专为中国用户优化的 AI Skills 社区。**

在整个 Trace 体系制定中，我和开发也反复打磨了很多个版本。Skill 要说简单也简单，它无非就是封装了一些场景使用定义规范、产品 API 的纯文本文档。但是在使用过程中，其实还有很多问题是用户关心或者在选择的时候没有考虑到的，**这里就需要作为平台方的我们，帮助用户提前判断，加速筛选：是否没有把渐进式披露考虑进去，导致让用户消耗 token 过大；用户在 Agent 中使用 Skill 的时候，往往自然语言 prompt 过于发散，Skill 是否能正确识别到用户的意图，并处理好能力边界.....这些都是用户的硬痛点。**

基于此，我们做了 TRACE 评测体系，把一个模糊的问题拆成五个独立的观测维度，来给每个 Skill 进行评测打分：

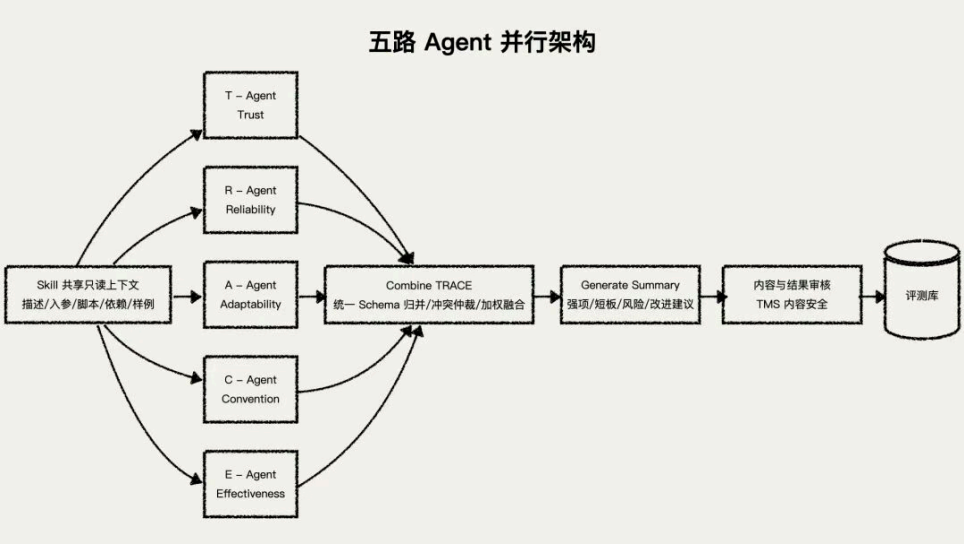
TRACE 维度	评测子维度	主要回答的问题
Trust（可信任度）	国内适配性、安全性扫描	关注元数据真实性、声明与实现的一致性、来源可信度以及安全边界。
Reliability（可靠性）	异常处理、功能完整性、运行稳定性	检查依赖完整性、异常处理、重试机制、超时表现和实现缺陷。
Adaptability（适用性）	能力边界定义、触发方式	评估参数兼容、异常路径、版本差异、环境容忍度与边界条件处理能力。
Convention（规范性）	反模式与 FAQ、文档质量、渐进式披露、结构清晰	检查文档结构、参数 Schema、目录命名和依赖声明是否符合工程规范。
Effectiveness（有效性）	输出准确性、内容完整度、创造力与增值、开箱即用度	判断产物是否正确、是否符合用户意图，以及 Skill 能否真正完成目标任务。

有了这份评测体系后，接下来的问题是：怎样能让平台里数万个 Skill 快速、稳定地经过这套检查呢？在这里，我们重点又建设了以下的基础设施。

2.1 从单体串行，走向五路并行

传统评测流程通常由单个 Agent 依次完成五个维度。流程容易理解，但每增加一个维度，都意味着更长的等待时间。当平台需要处理大规模 Skill 集合时，串行架构会迅速成为吞吐瓶颈。创作者肯定是希望上传 Skill 后能第一时间发布出去，而平台又希望 Skill 发布前就能够通过 TRACE 评测给 Skill 打分审核，因此，评测的用时就成为了首先要解决的问题。

我们后来把 T、R、A、C、E 拆成五条并行链路：它们共享同一份 Skill 上下文，各自完成判断，再统一做结构校验、冲突处理和结果汇总。在典型场景中，评测目标耗时由约 10 分钟缩短到约 1 分钟。每个维度也可以独立扩容和优化，新增规则时无需推翻整套体系。



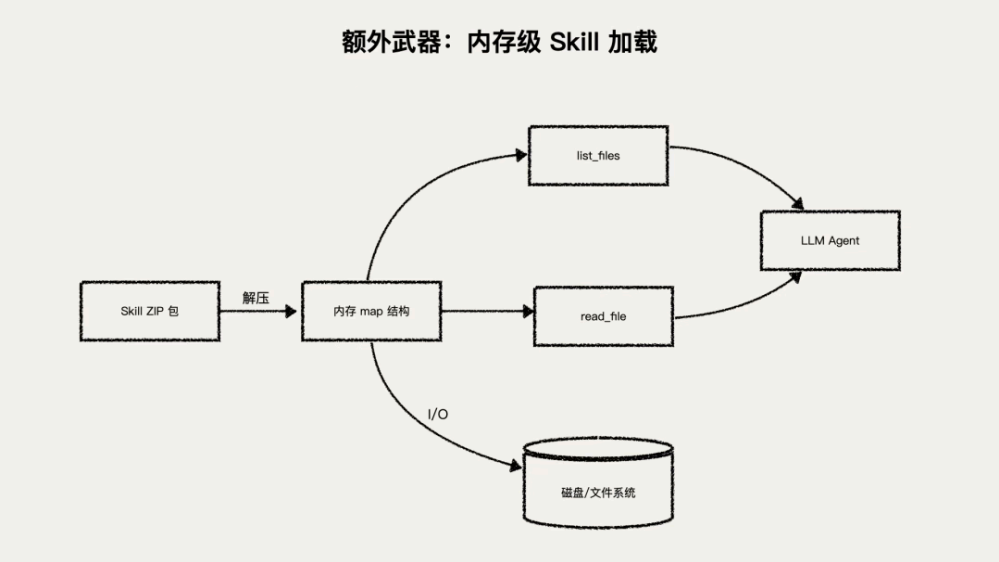
2.2 内存级加载，让 Agent 专注于理解 Skill

评测的另一个技术问题是：Agent 到底应该怎样读取一个 Skill。

如果五路评测各自解压、读取文件，就像五位检查员各拿一份复印件：速度慢，还可能出现版本不一致。我们将 Skill 包一次加载到内存，再通过 `list_files`、`read_file` 等受控工具按需读

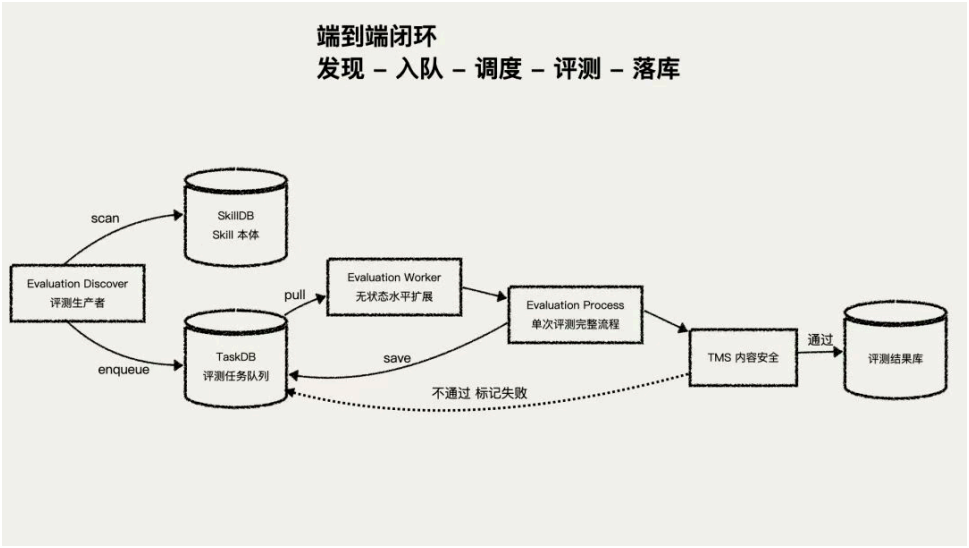
取。

这样既减少重复操作，也保证五个维度看到同一份内容；读取范围被限制在当前 Skill 包内，评测系统自身同样遵守安全边界。



2.3 给每次评测一张可追踪的“工单”

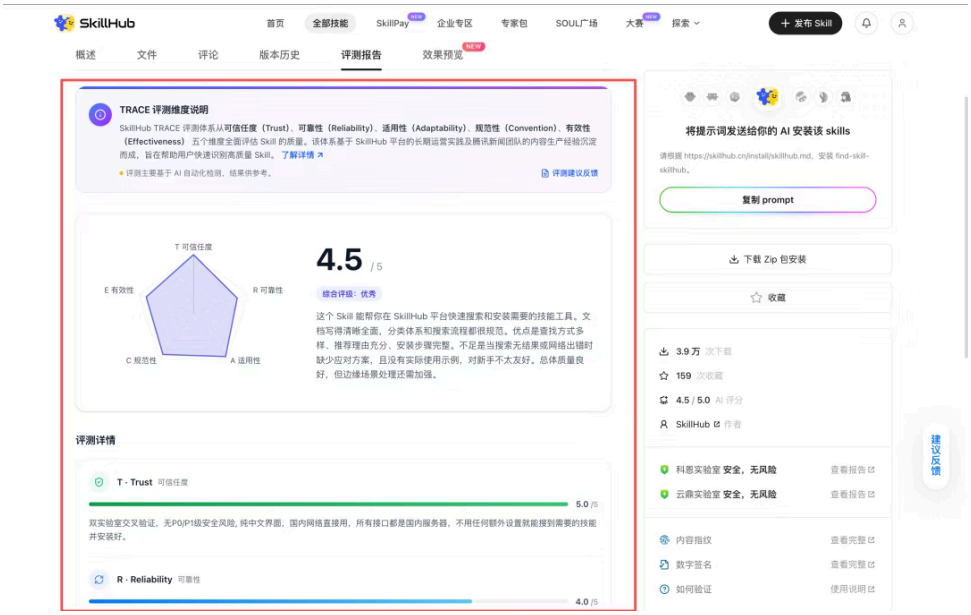
一次模型调用可以生成报告，却无法支撑持续运营。新版本需要自动复评，失败任务需要重试，异常结果还要进入人工处理。我们因此把链路拆成“发现—入队—调度—评测—审核—落库”：每个 Skill 都有自己的评测任务和状态，任何一步失败都能被记录、恢复和追踪。**这让 TRACE 从一次性的 AI 判断，变成了可以长期运行的质量生产系统。**



目前，我们基于 Trace 测评能力，对 SkillHub 上数万个 Skill 进行了逐一扫描，并为每个 Skill 生成简要评测报告，帮助用户快速筛选和甄别优质 Skill。

同时，我们还为创作者提供了更加详细的评测报告，并集成到创作者后台。创作者可以清晰了解 Skill 的整体表现，以及在各项细分维度上的具体得分和优化建议，从而有针对性地迭代和提升 Skill 质量。

这一机制不仅帮助创作者持续优化作品，也让平台能够沉淀更多高质量 Skill，最终实现消费者、创作者与平台三方共赢。



三、TRACE 看“体检报告”，但如何测试 Skill 是否真能干?

在前面的 TRACE 评测中，我们主要通过静态分析判断一个 Skill 是否可信，但它只能告诉我们 Skill“写了什么”，不能完全证明它“运行时会做什么”：

- 安装脚本无法执行，或者依赖版本不兼容；
- 说明文档声称支持某项能力，但 Agent 实际不知道如何调用；
- 可以生成结果，但结果并没有真正解决问题；

……

所以，我们又搭建了一座“封闭试车场”——云端隔离运行环境。**开发者可以直接在 SkillHub 中试跑 Skill，观察 Skill 在 Agent 里运行的效果是怎么样的，并且支持把有代表性的运行沉淀为详情页上的效果案例。**

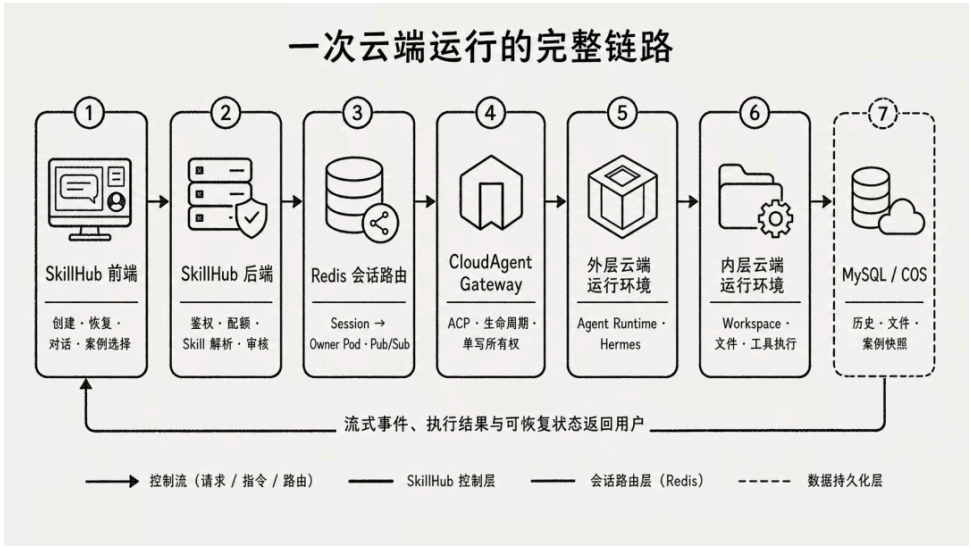


开发者可以在 SkillHub 中直接试跑 Skill，并观察真实对话和执行结果

3.1 一次试运行，实际经过了什么

云端隔离运行环境会解析目标版本，把 Skill 文件装进隔离 Workspace，再让 Agent 读取 `SKILL.md`，按说明调用脚本、资料和工具。只有环境返回 ready 信号，任务才会开始。这样可以排除一种常见的“假运行”：模型知道 Skill 的名字，也能生成一段像样的回答，实际并没有加载和使用它。

涉及文件读写、依赖安装和终端命令的操作，会进入独立工作区执行。平台业务服务与不可信代码彼此隔离，任务结束后还要完成清理和校验，才能安全复用环境。**复杂的隔离设计最终只为一个简单目标服务：用户看到的结果，确实来自这个 Skill 的真实执行。**



3.2 再让用户看懂过程，断开后也能接着跑

Agent 执行任务会经历分析、读取文件、调用工具、生成答案等多个阶段。我们把底层事件整理成用户能看懂的执行轨迹：**现在做到哪一步、工具是否成功、任务正常完成还是意外中断。**

用户因此可以判断结果怎样产生，也能快速定位问题。

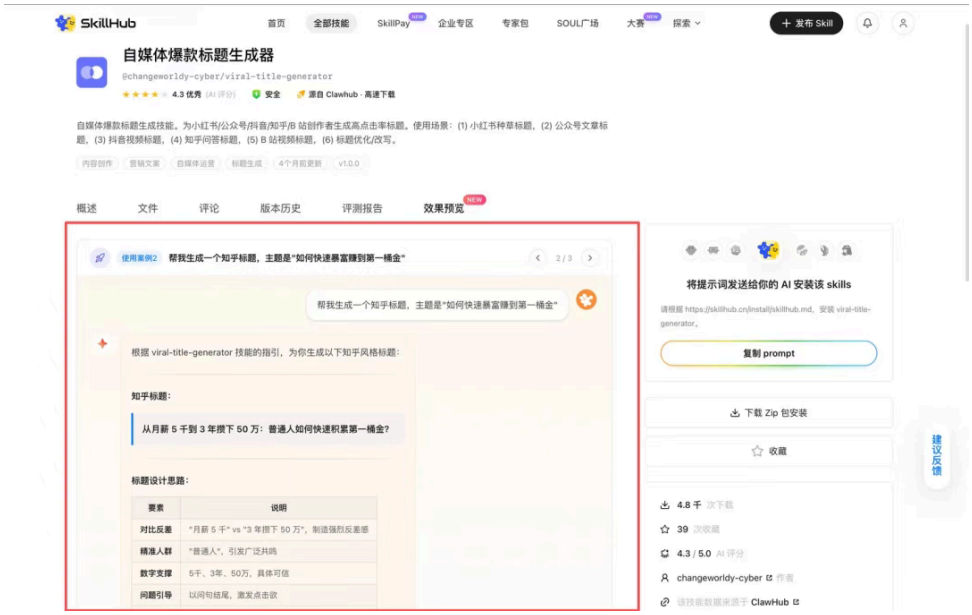
真实会话还会遇到网络中断、服务切换和环境重启。平台会保存消息、文件和关键运行状态，在条件允许时恢复原来的任务现场。这里最重要的判断很朴素：聊天记录还在，不代表运行环境还活着；页面显示“可继续”时，Workspace 和执行权限必须真的已经恢复。

3.3 针对运行环境的结果，我们沉淀了“展示案例”

目前我们计划把云端隔离运行环境的能力开放给开发者。一方面开发者上传 Skill 后，可以在平台内直接验证其 Skill 在真实环境中的运行效果，另一方面，我们也提供在运行环境中生成展示案例的能力，供消费者快速了解其 Skill 如何使用。

因此，SkillHub 把会话和公开案例设计成两个不同对象。**运行环境会话是实时的、可继续执行的，包含完整上下文和运行状态；效果案例则是用户从一次真实会话中挑选出的消息与文件快照。**

在前端，Skill 作者可以与 Agent 进行多轮对话，观察思考过程和工具调用，然后进入选择模式，挑选最能说明效果的回答，填写案例标题和补充说明，生成案例。



作者可以从真实的运行环境中挑选代表性问答，沉淀为 Skill 详情页可展示的效果案例

TRACE、云端隔离运行环境和效果案例，分别解决了三个不同层面的问题：

层次	回答的问题	形成的证据
TRACE 测评	这个 Skill 看起来是否可信、规范、可理解	结构、依赖、说明和静态扫描结果
云端隔离运行环境	这个 Skill 在真实环境里是否能够被 Agent 正确使用	执行轨迹、工具调用、工作区变化和最终结果
效果案例	这次成功运行是否值得被其他用户看见和复用	经筛选、审核并持久化的真实会话快照

四、有了质量底座，接下来如何让更多优质 Skill 获得曝光？

大多数内容社区都会使用点击、收藏、下载等行为做排序。这些能一定程度上反馈了内容的受欢迎程度，毕竟都是用户真实用脚在投票。但也正如前文所说的，SkillHub 跟普通内容社区不同的是，用户来这里不但只是为了看内容，他们的核心诉求是把一个 Skill 里面定义的一坨能力，能装进 Agent 中，完成真实任务。**因此，在做榜单的过程中，我们不但挖掘用户的行为数据，还结合了 TRACE 测评等维度，生成最终的榜单数据**：

榜单	想解决的问题
推荐榜	找到近期质量和数据都相对稳定的 Skill
飙升榜	捕捉时效性更强的新趋势
下载榜	保留最直观的长期热度入口
最新榜	给新品机会，同时过滤明显噪音

具体执行时，长期霸榜的“装机必备”类 Skill 已经能在下载榜充分体现长期热度，因此我们在榜单设计时，在推荐榜和飙升榜中适度降权，为近期优质供给腾出空间。最终，人工运营也始终存在，新品需要冷启动，垂直内容需要被“捞取”，热点需要专题承接，头部固化需要认为校准。规则、算法和运营共同把海量的供给，压缩成用户可以做选择的有限集合。



专题案例：与微信团队合作的微信小程序 AI 开发专区

如果说榜单像实时路况，标签更像是地图

刚开始做标签调研时，我看到几乎所有同类产品都有分类。就正如正文开头提到的这是社区常规手段之一，“别人有，我们也做一个”，但更深一步你就会去想：在 SkillHub，标签究竟要解决谁的什么问题？

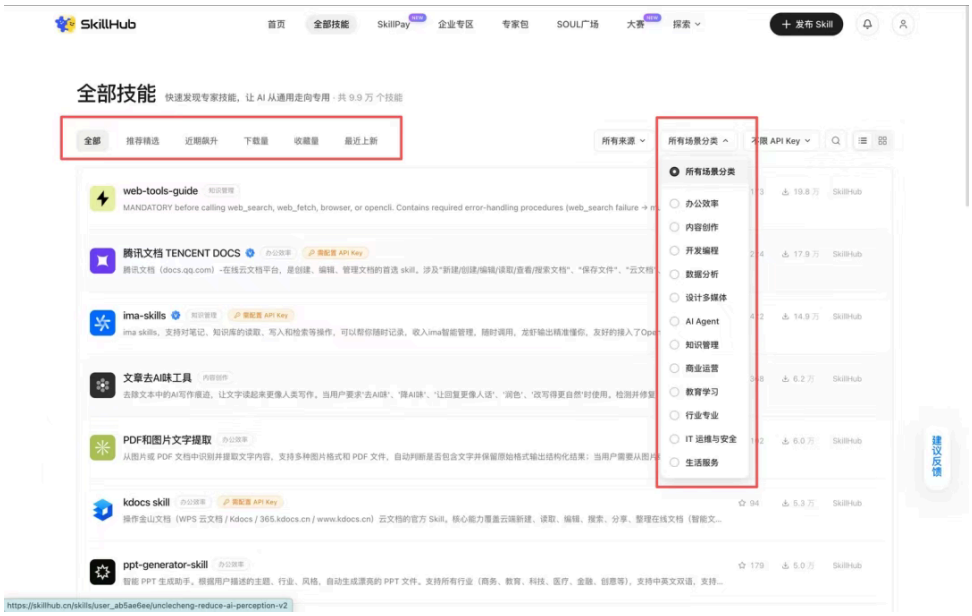
标签分类就更像是地图一样，把庞大的 Skill 世界划分成可理解的区域。我们一开始没有把标签做成一组任意扩张的词，反而是建立受控、分层的分类体系：

- 12 个一级主分类，回答“我该去哪里逛”；
- 每个 Skill 归属 1 个一级分类、1 至 3 个二级类目，回答“它具体能做什么”；
- 行业类目回答“它适合哪个行业”；
- “需要 API Key”等系统标签回答“我能不能直接用”；

这套体系也是经过了对 SkillHub 的用户洞察， 它从中文用户的任务语言出发，同时兼顾治理和运营。一级分类保持克制，避免导航无限膨胀；二级类目提供精度；运营标签不与长期分类混在一起；存量 Skill 先由系统自动打标，再逐步开放作者校正和后台治理。后续，我们也计划把社区内的搜索做成支持智能语言搜索，用 AI 解决用户使用 AI 的问题。

我们也看到了有一些 Skill 社区，没有制定统一的分类标签体系，而是让 AI 自己去做语义理解进行自动打标签。在效率优先的年代，这似乎是一个不错的解法，快速上线需求，无需思考设计。但在我看来，起码说在竞品的结果呈现上来看：平台的 Skill 就很多，标签体系又随意任意打，有一些标签我甚至不能理解为何也能成为标签，最终呈现结果上看，不但没有提高用户筛选合适 Skill 的效率，反而增加了用户的理解成本。

克制一点，由平台来统一定义标签体系，标签分类不宜过多。**现在，SkillHub已经可以按推荐、飙升、下载、最近上新等方式浏览，并结合来源、场景分类、是否需要 API Key 等条件筛选；详情页也会展示一级分类和二级类目。榜单、搜索、专题合集、大赛和后续的 Agent 检索，都开始说同一种“语言”。**



五、对人友好后，还要让 AI 找到 Skill

Skill 面向的对象是谁？如果帮用户找 Skill 的，不再是用户自己，而是 Agent 呢？在实际场景中，用户不但会到 SkillHub 站点里浏览、搜索 Skill，更会在 Agent 中直接让 AI 推荐发现 Skill。

目前最受海外大众认可的 Skill 开源平台——skills.sh，它有一个帮助用户发现合适 Skill 的「find skill」。它的逻辑很清晰：先理解任务和领域，优先查看热门榜单，再通过 CLI 关键词搜索；推荐前重点检查安装量、来源声誉和 GitHub star，最后向用户给出候选与安装方式。**skills.sh** 的搜索接口也会对单词使用模糊匹配、对多词查询使用语义搜索，榜单则主要依赖安装数据。

这个方法适合以 GitHub 和开发者生态为中心的供给。**但回到 SkillHub，我们面对的是大量中文自然语言需求、跨场景任务，以及已经建立起来的分类、评测和来源体系。因此我们没有只复制一个“搜索命令”，而是把一套中文 Skill 发现策略封装成了自己的find skill：**

1. 先从用户原话中抽取任务意图、领域分类和中英文候选词；
2. 对同义词、上位词做 2 至 4 组扩展，而不是只搜索一次原句；
3. 多次调用 SkillHub 列表接口，必要时叠加一级分类和属性标签，合并后按 slug（每个 Skill 的唯一标识） 去重；
4. 根据名称、中文描述和任务契合度重新排序，同一档位再参考下载和安装热度；
5. 只保留最合适的 3 至 5 个结果，说明匹配理由；用户确认后，再进入安装。

例如，“帮我自动写周报发给老板”不会只被当成一串关键词。Agent 会先把它理解为“办公效率”场景，再扩展“周报、工作汇报、日报、weekly report”等候选词。如果分类内结果太多，就继续收窄；如果没有结果，就去掉分类限制，换同义词和上位词放宽召回。

这也是 SkillHub 与一般 Skill 目录差异化的一点。目录解决收录，市场解决交易，社区解决互动；而一个面向 Agent 的 SkillHub，最终还要成为机器可以理解和调用的能力索引。**SkillHub 的 find skill 上线后，在没有任何推广的情况下，3天即登上了飙升榜第一，一周即破了 8000 安装量。**



在 Agent 中使用 find skill 来推荐相关技能

结语：如何找到那 20% 值得被看见的 Skill

回头看，我们做 TRACE、Skill 试运行、榜单、标签和 find skill，表面上是五项不同的产品能力，底层其实围绕同一件事：

在供给极度丰富之后，如何让用户更高效地找到平台中那些值得被看见的 Skill

用户需要降低试错成本，作者需要获得与质量相匹配的曝光，平台需要避免劣币驱逐良币，Agent 则需要把模糊任务稳定地映射到可执行能力。因此，我把 SkillHub 的推荐工作理解成一套“信任与分发基础设施”：

- 评测和试运行建立证据；
- 榜单和标签组织供给；
- 搜索与推荐分配注意力；
- find skill 把这套能力开放给 AI；
- 最终的真实调用结果，再回到评测和排序中。

这套体系目前也还在建设当中。评测还要持续适应新的 Agent 形态，Skill 还要补齐更完整的产物验证，标签也会随着真实需求继续迭代，创作者也需要更多曝光和激励，去支持平台做得更完善。但至少有一个方向已经越来越清楚：**当 Skill 从几百个增长到几万个，平台不能只是证明“这里什么都有”，帮助用户和 Agent 更快确认找到优质的合适的 Skill，反而显得更关键，也直接影响了平台对用户的价值。这或许是 SkillHub 做内容治理与分发最重要的产品观。**

欢迎前往官网体验：

https://skillhub.cn/



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号



阅读 2.0万

留言

写留言

还没有留言，来写首评